

Contents

The Suite	1
Part A — The Gates	2
A.1 The two readings, and the linkage law	2
A.2 The certificate engine	2
A.3 Datapaths, clocks and tokens	3
A.4 Parameters, and one promise across three machines	4
A.5 The miss queue’s own gates	5
A.6 The ladder	5
A.7 The language’s single sources	6
A.8 The checker	8
A.9 The compiler	8
A.10 The differentials, the corpus, and Lean	10
Part B — The Software Architecture	11
B.1 The picture	11
B.2 The modules	12
B.3 One entry, end to end — the artifact dataflow	13
B.4 How a solve is assembled	14
B.5 The contracts between the pieces	14
Part C — How to: installation, usage, and a first block	14
C.1 Setting up	14
Where it goes, and where you work	15
The short way	15
The two system tools, and why one of them is special	15
If you already have the tools	15
Then run something	16
C.2 Usage — the verbs, in the order a block meets them	16
C.3 A block, run for real: the miss queue	17
C.4 Upgrades, and reporting a gap	19

The Suite

Every guarantee this tool makes, how each one is held, and the software behind them. Written for a person meeting the system for the first time.

Part A is the trust story: the fifty-two automated gates that run on every release, each named, each with the sabotage that proves it can fail and the incident that created it. You do not run these — the maintainer does, before anything reaches you — but knowing what they check is knowing what a green certificate is worth.

How to read this book: if you are new to the tool, read `GETTING_STARTED.md` first — it is the guide for a hardware engineer meeting the route from zero, and it explains the ideas this book assumes. **Part C** here is the reference half of the same ground: installation, every command, and a worked example run for real. **Part A** walks the fifty-two gates in ten chapters, grouped by what they guard rather than by file order. Every gate entry answers three questions — *what does it prove?* *how would it fail if the property were broken (the sabotage)?* *what real event made it exist (the incident)?* — because a test whose reason is forgotten is a test

someone will one day delete. **Part B** is the software: the layers, the modules, the dataflow of one entry, and the contracts between the pieces. Two companions go deeper where this book points: `TOOL.md` (the v2 core file by file, written 2026-08-26, before the language front end existed) and `AUTOMATION.md` (the commands and the open tracker).

One principle governs Part A, stated once here: **a check must be able to fail**. Nearly every gate carries a *sabotage* — a deliberate breakage that the gate must catch — because a green check that would also be green on a broken system proves nothing. The suite’s own history is the argument: the vacuous cross-check, the empty-program “INDUCTIVE”, the reset-exempt monitors listed as proven while unfireable — each was a check that could not fail, and each cost real work to unmask.

Part A — The Gates

A.1 The two readings, and the linkage law

The route’s central discipline is that a specification watches the design through **windows** — derived views of the design’s own flops — and never keeps a mirror of the state (a “notebook”) that must then be proven consistent with it. These five gates hold that line.

- **lint_refuses_ungated_spec_ghost** — a v1-style spec (event-driven ghost state with no `refmodel` gating) is refused *by name* at the spec scale, with the linkage message. The FIFO’s v1 spec is the canonical wrong shape. Without this gate the route would quietly re-admit the notebook disease it was created to escape.
- **gating_is_per_rule_and_literal** — partial gating is not gating: a predicate with one ungated rule is `UNGATED`, and gating means the literal `refmodel` in the body — transitive gating is deliberately *not* inferred, because an inference is a place for a hole to hide.
- **bounded_only_classification** — a monitor whose body reads a gated predicate is classified `BOUNDED-ONLY` (it runs in the bounded legs, never in the step, where its rules would be inert and “inductive” a vacuous claim); one that does not is left alone.
- **gated_reference_model_is_allowed_and_reported** — the split end to end on a minimal design: a `refmodel`-gated reference model is accepted, its monitor is reported bounded-only, and the remaining set is proven inductive. Sabotage: drop the gate from one rule and the same spec is refused.
- **unit_scale_keeps_the_job_ghost** — `contract.py`’s standalone path is `UNIT` scale: the event-captured job ghost stays legal there and the unit proves inductive. This is the assume-guarantee leg — a unit’s contract may hold a small ghost precisely because the unit is proven alone.

A.2 The certificate engine

The runner (`refine.py`) is the machine that turns a contract and a design into a verdict. These gates pin what its verdicts *mean*.

- **stimless_certificate_path** — `refine(spec, None, level, induct=K)`: no stimulus exists; the runner builds its own reset base (inputs free) and runs the normal-form step. Sabotage: a design whose reset value violates a property fails the base.
- **ghost_free_induction_is_normal_and_fast** — the existence proof that induction is fast when nothing is mirrored: the `am2901`’s ghost-free spec is inductive at `K=1` in seconds. This is the performance thesis as a gate.

- **scenarios_directed_and_sabotaged** — the anti-vacuity half: a good scenario reports OK (the state is possible *and* the natural operation cannot be violated), and the two failure shapes are distinguished — an impossible situation names the contradiction, a defeatable expectation reports VIOLABLE with a counterexample table.
- **delivery_obligation_three_verdicts** — the owed-to-Lean protocol, all three verdicts on one fixture: a model matching the design’s term is discharged by identity; a symbolic difference is OWED to Lean (recorded, never miscalled a failure); a concrete mismatch is a violation.
- **failtype_vocabulary** — a spec written entirely with `failType(Name, T)` heads certifies exactly like `bad(Name, T)`: the tags join the property set, the step proves them, and a violation is caught and *named*. This is the vocabulary the language compiler emits, so the gate is what ties the compiler’s output to the runner’s input.
- **reset_less_block_is_judged_at_every_instant** — the runner’s first question, *can any instant be live?*, asked of the base’s own program and refused by name when the answer is no. Incident: a block with no reset compiled to a contract in which nothing derived `live(T)`; every monitor was guarded by it, so a plainly false property was INDUCTIVE and the only symptom was every scenario reporting “no compliant state”. Three halves: the compiler emits `live(T) :- gtime(T).` for a reset-less block (a block with a reset keeps the release-derived rule); on a real reset-less design the false property FAILS the base and a true one is inductive; and the sabotage — the live rule removed by hand — is refused before any verdict.
- **expand_handles_pack** — `expand`, the verb that shows a design in the emitted schema, accepts `pack(L)` by delegating to `compose`’s one expansion, and the expanded program agrees with the compact one instant by instant on the packed word. Incident: it raised `cannot expand operator pack` exactly when a diagnosis needed to read the model.
- **composed_temp_dir_does_not_outlive_the_run** — the composed program a lint writes is gone when the process exits (a subprocess lints a lane design and prints the path; the gate asserts it no longer exists). Incident: 3,735 of them in one user temp directory.
- **multi_bit_lane_round_trips_by_member** — a lane wider than one bit compares BY MEMBER on the translated sides: constants 2 and 5 come back as 2 and 5, a 1-bit lane still passes, and the projection leans on no `@func` the design never used. Incident (G27c, a silent wrong): the translator models logic `[1:0] [3:0] x` as flat bits and the harness read `x(0)` as member 0 — it is bit 0 — so every wide lane disagreed with itself; the first fix then dropped silently because `@add` was undefined on an add-free design.
- **roundtrip_solve_timeout_is_a_refusal** — a solve that does not finish is a TIMEOUT status naming the limit, never a traceback, and every solve in the round trip and the certificate runner hands `clingo` its own `--time-limit`, so a solver stops by itself even when the Python that spawned it is killed. Incident (G27b): the 256-cell grid ended in an uncaught `TimeoutExpired`; nine orphaned solvers, 5 GB, were found on one machine the same day.

A.3 Datapaths, clocks and tokens

Data is never enumerated; time can be design-computed. These eight gates guard the two readings’ machinery.

- **opaque_datapath_certificate** — the `opaque_datapath.` directive: control solves treat internal data nets as per-instant tokens (an enable-gated hold/load fork would otherwise multiply grounding candidates), while the delivery obligation still computes the real term. A `compose` regression is asserted too — the flag once died in `compose`’s explicit field copy.
- **opaque_datapath_does_not_mask_control_bugs** — the soundness half: severing the datapath must

leave the control checks intact; a conveyor whose valid flop loads constant 1 still fails under the directive.

- **opaque_datapath_refuses_data_memories** — scope honesty: a data memory under the directive is refused by name, never silently mishandled.
- **derived_clock_semantics** — a register clocked by a design-computed net updates exactly across that net’s rising transitions on the master axis and holds elsewhere. The toggle-divider fixture ends at 4 over eight master cycles; the free-axis wrongness this extension replaced would end at 8.
- **derived_clock_refused_on_memory** — scope honesty again: a memory on a derived clock is refused by name.
- **symbolic_equality_is_transitive_for_cams** — the equality theory closes transitively across every chain the program compares, so a match vector is always consistent with *some* concrete address assignment. Three solves discriminate it behaviourally (the pre-theory library passes none of the UN-SAT legs), including the miss-queue shape: one request matching two slots held distinct is impossible, matching one stays possible.
- **symbolic_gate_equality_link** — the gate spelling of a comparator, `rnor(bxor(A,B,W),W)`, is pinned to `eq(A,B)`, licensed by the Lean lemma `RouteLean.Cam.comparator_is_equality` proven for all widths. Forcing the spellings apart is unsatisfiable; agreement both ways is satisfiable.
- **data_on_a_lane_declares_every_member** — `data(x)` on a LANE makes every member a token net. Found by the regeneration run, whose slot payloads are lanes of tokens; without it, lanes and the symbolic reading could not coexist.

A.4 Parameters, and one promise across three machines

- **parametric_module_with_mrd** — a parametric module reading a flop array through `mrd` loads and lints: the term resolver must not mistake the array’s *name* for a parameter (the subset checker had the exemption; the resolver’s copy was missing).
- **parametric_print_farray_declaration** — printed storage of a parametric module is declared over the *parameters* (`logic [WIDTH-1:0] m [0:DEPTH-1]`), never the resolved defaults — the user’s catch on the v2 FIFO: parameters real everywhere but fake on the array meant an override would have silently built wrong hardware, and the round trip (which runs at defaults) cannot see it.
- **multiplier_contracts_are_rule_identical** — the comparison family’s doctrine as a gate: `wallace32`, `booth_wallace32` and `booth_production32` carry rule-identical `spec.lp` files. One promise, three machines — if any contract gains a rule the others lack, the PPA comparison silently stops being about the machines.
- **lane_index_may_name_a_parameter** — a lane’s neighbour offset and wrap block may each be a `PARAMETER`: a 3×3 torus unrolls to the right members at two sides, prints `q[((i + (CELLS - SIDE)) % CELLS)]` with no residue baked from one size, has an identical generate body at every side, reads a parenthesised index as the index, refuses an unknown name by name, and round-trips against Icarus. Incident (G28): the wrap index existed and nothing shipped said so; the only spellings accepted were literals, so a generator unrolled 256 cells and print parity differed in 4,268 lines.
- **lanes_have_axes** — a lane with two axes: the 3×3 torus unrolls to the right members at two sides, prints `logic [SIDE-1:0] [SIDE-1:0]` with one nested generate per def and an identical body across sides, pack flattens row-major, a one-axis def on a two-axis lane is refused by name, the round trip agrees with Icarus on a `POSITIVE` number of samples, and a bench whose output lines match nothing `FAILS` (sabotage). Incident: the first run agreed on zero samples — the bench spelled `y(0, 0)` with a space — and said so as if it were agreement. The same run found F34, F35 and F36 in the translator (the sweep row nested_generate_computed_index_on_a_2d_packed_port carries the first two).
- **lane_instances_have_axes** — the sequential half: `inst_lane(uG, ff, (n, n))` gives instances

`uG(r, c)` with member-to-member pins, prints a nested instance `generate(g[i][j] <= gM1[i][j])`, round-trips the 4×4 flop bank against Icarus, round-trips Conway’s Life at side 3 (eight per-axis wraps, the count, the rule, the grid in the flops, the load path) with the print identical at side 4, and refuses by name `arch_reg` with a dimension list and a pin between lanes of different rank. Incidents on its first run: F38 (the power-on of a nested-generate register spelled flat, the bank dark from $T=0$ in both modes) and, under the sweep’s random stimulus, F39 (`a[c]` under `for (r) for (c) read the row`) — the rows `register_bank_in_a_nested_generate` and `transpose_in_a_nested_generate` carry them.

- **bit_position_over_the_loop_variables** — inside a lane `def, bit(data, add(mul(R, side), C))` reads a flat port at the cell’s own position: members at two sides, the print `data[((r*SIDE)+c)]` identical across sides, the Icarus round trip, and the two refusals (a variable that is not the `def`’s; a loop variable outside a lane `def`). Incident (G30): a grid could not read its flat seed port per cell without unrolling.

A.5 The miss queue’s own gates

The worked entry carries the gates that prove *it* — and, since the regeneration, the gates that exercise the certificate’s full modern form.

- **rv_missq_certificate** — the archived first entry’s certificate (kept as a fixture gate): promise families inductive at $K=1$ with zero hand-written invariants, scenarios reachable. Sabotage: undo the same-cycle-fill exclusion and the step fails on exactly the accept/merge monitors.
- **rv_missq_composed_certificate** — option 3 on that entry: the queue with abstract slot instances, the slot’s contract assumed, its requires proven as queue obligations. Sabotage: break the allocation priority chain in the glue — the only logic the composed proof still owns.
- **abstract_composition_assume_guarantee** — the tool half of composition: an abstract child’s contract is assumed in the stimless certificate, its requires checked, and the empty-state diameter shortcut is gated — it once declared **INDUCTIVE** without the step running, a false proof this gate makes impossible. Teeth both ways: the parent property is provable *only* from the assumed guarantee, and a parent that misdrives the child fails on the require.
- **missq_cam_standalone** — the parameterized CAM proven as a datapath problem: gate-level XOR–NOR cells against a contract in ideal equality, bridged by the gate-spelling constraint. Sabotage: an AND-reduce cell fails.
- **rv_missq_fetch_queue_certificate** — the regenerated entry’s **certificate as data**: the `verify.json` manifest runs the standard half (the NOT EXERCISED bucket asserted), the strong half (`--free-reset`, judged by the induction alone, the reset monitors required bound), the FV parity producer, and the discriminating second configuration (green at depth 2, *and* its contract must reject the default design). Runs on a copy so the suite never rewrites committed logs.

A.6 The ladder

- **ladder_gates_each_step_on_the_previous_approval** — a step cannot begin until the one before is approved, and the tool has no path that sets approved itself. Building the signature before the specification is approved is refused by name.
- **ladder_approval_goes_stale_when_the_artifact_changes** — the guarantee with real teeth: an approval records the digest of what was approved, so editing the artifact afterwards re-opens the gate — the innocent failure (“approved, then quietly improved”) caught mechanically.
- **ladder_refuses_a_malformed_file** — unknown keys, steps, states and out-of-order steps are hard errors: a typo silently ignored is a gate that silently did not run.

A.7 The language's single sources

- **dsl_grammar_has_one_source** — the grammar exists once, in `lib/dsl/grammar.ebnf`: the methodology renders it, the parser is built from its core slice. The drift it guards is the kind nobody watches for — each edit locally reasonable, the pair disagreeing with no symptom until a specification is accepted that the document forbids.
- **dsl_grammar_drift_is_detected** — the sabotage for the check above: change the source and the drift must be reported, or the check could be comparing a thing with itself.
- **surface_grammar_single_source** — the surface's half of the same doctrine: the condition patterns live in the grammar file's surface section, the desugarer *compiles its matchers from them*, Chapter 35 renders them, and every pattern's `ex:` lines are executable witnesses. Four claims, each with its sabotage: no drift; every example reproduces through the derived matchers; an unknown production refuses to bind; a rendered-block edit is reported. *One half of the surface is honestly outside this doctrine.* The CONDITION patterns are derived from the file, in both directions; the surface's EFFECT shapes — accept, create, choose, send, the `@next cycle` forms — are documented in the same file and dispatched in code, so that half is a description which can drift, and none of these gates would catch it. Extending the derivation to the effects is tracked in `AUTOMATION.md`. Stating the limit is the point: a reader who assumes the whole surface is gated would trust the wrong half.
- **pack_names_a_lane_as_one_word** — a 1-bit `net_lane` could not be assigned to a same-width port: the lane exists only as its members, the bare name is not a net, and there was no concatenation over a lane. The forced workaround — a weighted-sum chain — was a fake parameterisation, the defect the route had just built print parity to catch: **a missing spelling does not leave a gap, it forces a workaround, and here the workaround was the defect the checks were hunting.** `pack(L)` is explicit rather than a bare lane name accepted where a net is expected, because a lane of WIDE elements has a packed word too, and its cost is exponential in the element count — declared and budgetable, not inferred and discovered. Expanded for the solver (lanes are unrolled by then); kept for the printer, which prints it as the lane's own name, parametric by construction. Verified on the reporter's design: certifies with `byteMatchesCapturedBits` intact, prints `assign out_byte = cap;`, print parity passes.
- **changes_ledger** — `docs/spec2rtl2/CHANGES.yaml`, one entry per tool change with its reason: open entries are the worklist, fixed entries the changelog. The gate is what makes it more than a list: every fixed entry must cite a test `pytest` actually collects (a sweep row as `sweep:<row>`), every entry must carry a why, and fixed-ungated is allowed only as an admission listed here as owed. A citation to a test that does not exist fails the build — verified by sabotage — and the shipping gate asserts the file ships, because the first time it was built it did not, while its commit said it did.
- **print_parity_catches_a_module_parametric_in_name_only** — the check that was pointed at the wrong artifact. The route already regenerates at a second configuration and certifies there, on the principle that a module parameterised in name only dies at the off-default point. A block passed that at `dataBits=3` while its PRINTED module carried eight hardcoded `assign byteUpTo[i]` lines: below 8 an out-of-range reference, above 8 undriven bits. Both facts were true — the generator loops, so the ASP is parametric, and that is what the second point certifies; the parameterisation is lost in the print. The invariant was MEASURED rather than assumed: two prints of a genuinely parametric design differ only in the parameter defaults (the miss queue at depth 4 against depth 2 — four lines, every one a parameter $X = N$). It found the real defect on the real block and named the five lines that vanish. **The generalisable rule: name the artifact a property is about, and make the check read that one.**
- **an_enum_type_is_named_apart_from_its_net** — named states. `enum_member(st, ...)` printed a typedef NAMED `st` beside logic `[1:0] st;`, and iverilog refused the file. The named-state path otherwise existed — for net: an enum-typed intermediate prints with its type, a ternary with a cast, a compare by member name — but `arch_reg`, the v2 spelling for state the specification names, never ac-

cepted enum(E), so a block whose states have names carried encodings in its RTL. Now `arch_reg(st, enum(st))` is accepted, the type prints as `st_t`, the register resets to a bare member label, and the gate runs iverilog on the print because “compiles” is the claim — with the collision restored as the sabotage. The first enum register through the v2 round trip also found the round trip’s label mapping keyed on the enum-typed width alone, and a translator defect (F31, open) that the typed register sidesteps.

- **a_property_may_hold_always** — `@always <condition>`, the eighteenth structural keyword. The core had `always <expr>` from the start; the surface reached it only inside an `@every ... :` block, so a top-level “the state is always one of the three” needed a tautological trigger that lowers correctly and tells a reader nothing. Explicit and sigiled rather than “a bare condition means always”; the bare word is refused with the spelling.
- **a_module_name_may_need_the_esp_escape** — a block whose SystemVerilog name starts with a capital. ASP reads that as a variable, so the bare form is rightly refused and quoting is the only escape; the quoted tuple then reached the printed RTL as `module ('str', 'TopModule')`. The half worth gating is the SEAM: two subsystems solve this differently and neither is wrong — the authoring side quotes, the translator normalises by lowering the capital, because it must turn arbitrary SystemVerilog names into clingo constants and cannot quote every one. The round trip is where they meet, and using the authoring convention there left the modular half deriving NOTHING — which reads exactly like a design that does nothing rather than like an error. Sabotaged both ways, including the mistake actually made.
- **the_schema_documents_what_the_tool_accepts_and_emits** and **the_schema_covers_every_family_the_corporation** — the third single source, and the one that was missing longest. A model in the loop WRITES the design and its linkage and READS the generated contract; all three are ASP in a fixed schema that was written down nowhere, so an author inferred it from one worked example. `sv2asp2 schema` prints it, DERIVED from the tables that define the behaviour — `load.FACT_PREDS` (what the parser accepts and refuses everything outside of), `model.CELLS` (a primitive’s pins), and `emit._ROLES` (how an auxiliary atom is named) — so it cannot describe a language the tool does not accept. What is not derivable is the prose, so the first gate requires a gloss for every fact and every cell: a new predicate cannot ship undocumented. The second holds the contract half to reality — every predicate AND every verdict name in the miss queue’s compiled contract must fall into a documented part, and it is checked non-vacuous (the corpus must actually exercise the Wrong, Disturbed and NotSingleValued shapes). Two sabotages: a gloss removed, and the role table shrunk so emitted heads stop being classifiable.
- **cnl_structural_keywords_require_their_sigil** — the sigil rule: the eighteen structural keywords carry their @ so a reader sees the sentence’s skeleton and a typo fails *as* a keyword; grammatical words stay plain English; the sigils strip before desugaring, so the core is untouched.
- **signature_schema_accepts_the_entry_and_refuses_by_name** — the signature is the compiler’s symbol table, so a half-understood one is worse than a missing one. The role case matters most: a 26-bit address read as numeric is 67 million grounder values instead of one equality bit — a failure that would otherwise surface as a performance mystery days later in a different file.
- **the_signature_refuses_an_out_of_domain_reset_field** — role was refused against its enum and the reset’s fields were not, which is the asymmetry rather than an oversight in one field. `polarity` is CONSUMED: it decides which way `disable iff` runs for every monitor, so a near-miss (`active-low`, `activeLow`) fell through to the active-high default and every claim was enabled exactly where it should have been silenced. A wrong role is slow; a wrong polarity is a different specification that still certifies. The three fields the compiler does not read yet are validated too — an unvalidated field that is not consumed is a trap waiting for the release that starts consuming it.
- **the_documented_signature_example_loads** — the methodology’s own worked signature is RUN, and an example that does not load is a build error. It exists because the governing document showed

reset: <signal> activeLow once, a shape with no metadata and no clocks_and_resets and a polarity spelled the way the tool does not read: a reader following the page wrote something rejected outright, or — for the polarity alone — silently taken as active-high. Same doctrine as the grammar’s drift gate, applied to the schema: the page and the parser stay in step because a build says so.

- **signature_requires_the_wires_its_protocol_promises** — a readyValid interface with no ready port could never accept; a validOnly one *with* a ready port could refuse, which is exactly what validOnly promises it cannot do.

A.8 The checker

- **dsl_checks_pass_on_the_real_specification** — the floor: the entry’s own specification is clean under every check; a checker never run against a real file is a checker nobody has tested.
- **dsl_checks_catch_what_they_are_for** — one sabotage per rule, each a defect that has *actually occurred* (three in the entry’s own file). The scope and shadow cases matter most: both produce a specification that compiles, certifies, and means something other than what it says — no solver reports either, because the resulting contract is perfectly satisfiable.
- **dsl_lifetime_check_wants_exists_at_the_later_instant** — a claim speaking of an entry at a later instant must say whether it still exists, or the entry that *replaced* it can discharge the obligation.
- **dsl_lifetime_rule_spares_index_domains** — an @index domain has no lifetime; asking whether an address “still exists” would make every storage claim report a defect that is not there. Sabotage in the other direction: remove the @index declaration and the same file must be reported.
- **dsl_checks_pass_on_the_fifo_too** — the second block in the language, kept partly *because* it is not the miss queue: no objects, no CAM, pointers where the queue has slots. Writing it found the inverted-sense ready wire and the missing pointer vocabulary.

A.9 The compiler

- **dsl_emitter_is_general_not_fifo_shaped** — THE GENERALITY GATE. Both blocks must lower, and the second is the one that matters: an emitter built against one block alone passes while being shaped entirely by it. Measured when written: the miss queue went from 8 rules and 13 refusals to 69 rules and one refusal — and the one that remains is *correct* (s_eventually is an obligation, not a rule). Written as a floor, not an equality, so a better emitter is not a failing one.
- **dsl_emitter_reports_the_windows_it_demands** — the mount manifest: quantifying over entries silently asks the design which slots are live, a demand appearing nowhere in the source text; the emitter names it, so an author mounts windows instead of discovering an unfireable monitor.
- **a_bit_of_a_port_binds_its_position** and **compile_grounds_what_it_emits** — a contract that compiled clean, reported nothing, and could not be GROUNDED. Relating a multi-bit port to per-bit state lowers the bit to a boundary, and that declaration is a separate rule from the claim: the position was bound in the claim and free in the declaration. Clingo does not skip an unsafe rule — it stops grounding and takes the whole program with it, so nothing downstream of the contract could run, while the compile that produced it exited 0. The corpus could not have caught it: rvMissq emits no boundary(bit(...)) rule at all. The structural half is the second gate — **compile now grounds its own output** — and it is cheap because safety is syntactic, so the contract grounds with no design (the corpus compile is 0.4s in total). Two sabotages: the position unrecorded (the emitter must refuse by name, never emit unsafe) and the check wired to report (compile must fail on it, or the check could find the error and be ignored).
- **dsl_emitter_lowers_the_fifo_and_the_result_grounds** — grounding is the floor, not the goal: a contract can ground and still say the wrong thing (the differential’s business), but one that does not

ground says nothing — and two of the emitter’s first three defects were exactly that (an instant left unbound by a negation-only helper, and by a reset-exempt guard).

- **dsl_emitter_refuses_what_it_cannot_lower** — a construct outside the stage is refused by name, never guessed at: the refusal is what keeps a partial compiler honest.
- **compile_cli_end_to_end** — the `.cnl` → `core` → `contract` chain as one command: the corpus compiles, the contract carries monitors, the by-design refusal is *printed*, and the committed core is not disturbed.
- **generated_contract_wellformedness_check** — the collision guard: the emitter refuses its own output if a helper negates its own head or one name is defined by two lowerings. The incident: a renaming let a fresh helper collide with a reserved main name — caught once by reading; caught every time since by this.
- **a_declared_window_is_framed_whatever_its_shape** — THE FRAME RULE IS NOT ABOUT OBJECTS. A behaviour lowers to an event monitor plus a hold-otherwise frame monitor, and that second half is what makes a set of behaviours a complete transition relation rather than a description of some moments. The rule had been implemented three times over against the one block available and keyed by an object every time, so a block whose state is a phase, a counter and an indexed array got **none** — with no refusal, no missing-monitor count, and a green certificate, while nothing forbade the design from changing a captured bit between the cycle it arrived and the cycle it was presented. The gate asserts all three key shapes, and asserts the converse too: a window the specification only READS is a derived view of the design and must NOT be framed.
- **a_window_is_single_valued_and_it_is_checked** — what makes every other claim mean something. A claim lowers to “SOME value of this window is x”, so a window holding two values SATISFIES a claim the design violates — masking, not false alarms. It was true in the corpus by accident (each window mounted from one `val/3` atom, which the translator makes single-valued) and is now stated and checked. The gate proves the masking on the solver: with both one-hot bits high the machine is in `idle` and “done implies presenting” passes anyway. Deliberately a MONITOR, not an integrity constraint — a constraint would exclude the multi-valued runs, so a genuinely multi-valued linkage would come back UNSAT, which reads as “no counterexample”, which is how two translator defects once hid.
- **the_keywords_command_prints_the_vocabulary** — the sigil rule’s discoverability. The skill and the methodology both cited a command that printed the vocabulary, and for a while it did not exist. That is worse than an undocumented command: a reader who types it and gets `invalid choice` stops trusting the page. Both lists come from the grammar file, so the printed vocabulary cannot drift from the rule the desugarer enforces.
- **a_frame_catches_the_position_it_did_not_license** — its teeth, on the solver rather than in the text. A frame that emitted the right atoms but licensed every position would read correctly to a reviewer and forbid nothing, which is the same green-for-the-wrong- reason shape the frame exists to prevent. Two traces: the design changes the position it was licensed to write (must be allowed) and a different one (must be a named failure).
- **the_reset_exemption_follows_the_meaning_not_the_spelling** — Chapter 33 exempts a property that NAMES the reset from the file’s `disable iff`; what was implemented was one PHRASE that reaches the exemption by another route. The two coincide on the corpus. The same requirement written the other way compiled into a monitor needing the reset both asserted and released at one instant — present in the contract, counted among the monitors, unable to fire under any execution. **A monitor that is green because it is dead is worse than a missing one**, because a missing one shows up in a count. The gate checks the guard on both spellings, on a behaviour as well as a property, and checks that an ordinary claim is still silenced during reset.
- **a_scenario_may_quantify_over_a_declared_domain** — the same construct reaching one declara-

tion kind at a time, for the third time, and this one arrived as a CRASH rather than a refusal. The lowering is what the gate is really for: `scenario()` folds every binder into the situation EXISTENTIALLY, which is right for some and would have made each `bit J` mean “there is a position where the expectation holds” under a name promising every position — a check claiming more than it tests, which this route ranks below an honest gap. So the universal reading is asserted on the solver: all positions good reaches the scenario, one position bad does not, and the sabotage is folding it existentially.

- **a_property_may_speak_of_the_next_cycle** and **a_behaviour_may_quantify_over_a_declared_domain** — the two halves-of-a-construct. Each existed in one declaration kind and not its sibling, which is the shape this route keeps paying for: the language looks like it covers the case, and the gap only appears at the sentence nobody could write. Together with the exemption they are why “after a reset cycle the phase is idle” had no sound spelling at all — the exempt form could not carry a next-cycle clause, and the form that could was not exempt.
- **port_reads_share_one_sample** — a claim that reads one port several times samples it ONCE per instant: the Holds rule carries one sample variable and both `pval(bit(...))` reads; the true two-read bound is inductive and the false one is refuted, so the rewrite changed the join and nothing else. Incident: k fresh samples of a 9-bit port joined 512^k in the grounder, and a 3×3 grid’s neighbour count never finished — measured through the tool, two reads went from killed at 90 s to 9 s.

Two Icarus-arbitrated rows in the translator’s idiom sweep belong to this route’s third block: **ternary_assign_with_a_boolean_arm** and **nested_ternary_with_a_boolean_arm** (G27a) — a 1-bit ternary whose arm is a comparison or a logical op, refused as `word expr BinOp` in both modes because F29 had sent compound arms to the word cascade; such an arm is now a named bit. And **test_f32_word_bridge_budget** with the row **packed_2d_constant_element_select_above_budget** (F32): a 24-bit register-derived per-bit signal read only by constant element selects is not assembled into a word and its slices read bits (both modes, the same text modulo the instance prefix), grounds in seconds under a free power-on, keeps its word with a named warning when some rule reads it whole (promoted by `--strict-warnings` in both modes), agrees between the modes on every observable, and comes back to today’s shape when the budget is raised above the width (the sabotage). Incident: a 36-bit count array’s bridge never finished; the first fix refused statically and broke a 32-bit example that grounds fine with pinned inputs.

A.10 The differentials, the corpus, and Lean

Two rows of the translator’s Icarus-arbitrated idiom sweep belong to this route’s record, because the route’s own printer produces the shapes and its round trip is what found them:

- **sweep:ternary_assign_with_a_compound_arm** (F29) — the hold/set staging shape `assign qM1 = rst ? 1'b1 : ((q & a) | b);` was refused outright, so a one-hot FSM could be printed and not translated back. Fixed by delegating the last case of `_bit_read` to the general lowering it already used for slices. Loud before, correct after — and the fix removed the refusal that was masking F30.
- **sweep:generate_ternary_reading_lanes** (F30) — a per-lane ternary hoisted out of its generate lost the genvar: `idxHot(I-1)` with `I` unbound, `cap[i]` read as the word. Wrong rules at exit 0 once F29 let the arm lower; the round trip hung on the $2 \square \square$ join rather than reporting a mismatch. A temp hoisted inside a generate that reads a lane is now a lane by construction on every path — the F17 rule — after a refusal on the same shape over-refused `bob_demo`, whose temps were already right by the classifier’s closure. The round trip went from a 300 s timeout to Icarus agreeing on all 1300 definite samples.

The correctness bar for the compiler: never agreement with a hand-written artifact — two independent semantics, and agreement between them.

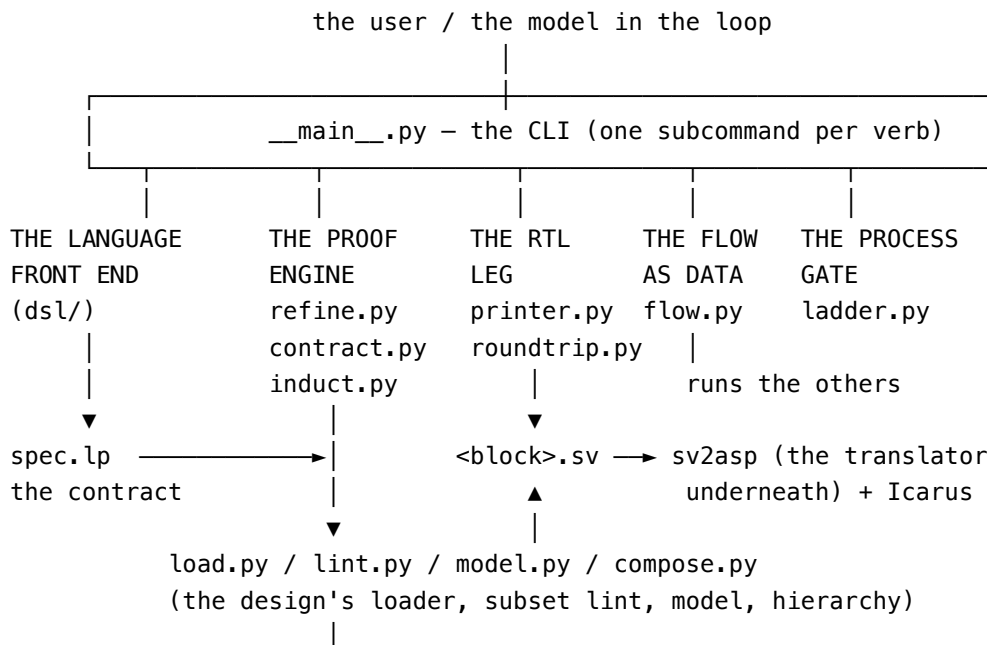
- **dsl_verdict_differential_on_the_fifo** — the generated and hand-written contracts must agree

on *verdicts*, never text (the retired version-3 bar): the real design certifies under both; a stuck valid flag fails both; a frozen write pointer fails both. The two vocabularies never meet — the linkage mounts both onto the same flops, and derived views of one register cannot disagree.

- **dsl_verdict_differential_on_the_missq** — the same, on the entry this work is for, with the sabotage being the real defect the entry’s certificate found (fillhit ungated from fillValid). Reaching green took three recorded diagnoses — double-shifted helper instants, a safety binder demanding existence where the claim said “gone”, and a contract compiled at depth-8 defaults judging a depth-4 design.
- **cnl_corpus_gate** — the corpus gate: the user’s structured-English draft, compiled, produces the same certificate verdicts as the symbolic specification — the real design green, both sabotage families red. Two notations, one block, one truth.
- **cnl_traceability** — the gate on the chain’s first arrow, which no theorem reaches and a table therefore must: every clause row names declarations that exist (or carries an honesty marker — SIGNATURE, WITHDRAWN, NOT TRANSCRIBED), and every declaration is owned by some clause — a claim no sentence owns is a claim nobody asked for.
- **stage5_reference_interpreter_differential** — the second semantics: dsl/interp.py evaluates the core’s property claims and lowered assumptions over concrete traces, from the methodology’s stated meanings, and must agree with the generated contract under clingo on admissibility and every (monitor, instant) verdict — with *coverage* (every monitor fired at least once, or the batch checked nothing) and two mistranslation sabotages caught (a dropped monitor; a bound off by one operator).
- **stage6_claim_schemas_proven_in_lean** — what each claim lowering *means* lives in Lean: the monitor schemas proven faithful to their trace denotations, the window’s determination instant a theorem, and the two natural mis-lowerings refuted by countermodels. The tie to running Python is the Stage-5 differential; this gate checks the theorems exist and the library builds.

Part B — The Software Architecture

B.1 The picture



▼
lib/aspfirst/*.lp + libgen.py (@func script)
the primitive library clingo actually solves with

Three facts organize everything. **The contract (spec.lp) is the hub**: the language front end produces it, the proof engine consumes it, and nothing downstream cares which producer wrote it. **clingo is the only judge** in the proof leg; Icarus is the only arbiter in the RTL leg; the two legs never share a verdict path, which is what makes the round trip evidence. **Single sources are load-bearing**: the grammar file feeds the parser, the desugarer *and* the documentation; the primitive library feeds every solve; a second copy of either would be where drift lives.

B.2 The modules

The core (src/sv2asp/aspfirst2/):

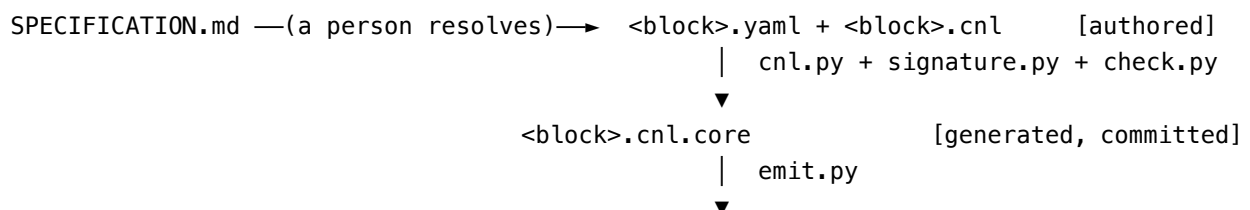
module	its one job
__main__.py	the CLI: compile, lint, refine, certificate, verify, print, roundtrip, ladder, contract, export, expand — argument parsing and dispatch, nothing else
load.py	the authoring subset's loader: facts (net, def, inst, pin, lanes, params, data, arch_*) into the Design model; lanes unroll here; parameter expressions evaluate here; everything outside the subset is a named SubsetError
model.py	the Design/Inst/Port data model and the primitive/@func/operator tables
lint.py	the static lint over a loaded design (combinational loops, widths, undeclared reads) and clingo_bin() — the tool-resolution seam (sv2asp.toml → env → PATH)
compose.py	hierarchy: child modules flattened or held abstract (contract-only), port bridging, clock/reset hints
refine.py	the certificate engine : the stimless path (reset base + normal-form induction step + scenarios + delivery obligations), the strong half (--free-reset), the reset-exempt NOT EXERCISED bucket, the owed-to-Lean protocol, counterexample tables
induct.py	the induction step's plan: which state frees at T=0, which inputs stay free, how the hypothesis is asserted
contract.py	the unit scale: a module proven standalone against its own contract (assume-guarantee's other half)
flow.py	the flow as data : verify.json executed by one runner — refine chains, induction_only, per-step logs, second_points with the discrimination check, round trips; unknown keys are hard errors
ladder.py	the human-gated process: seven steps, digests, staleness; no code path writes approved
printer.py	ASP → SystemVerilog under §27.1's conventions: parameter port lists and derived localparams, grouped generate blocks, inlined width-safe expressions, the xxM1 flop idiom

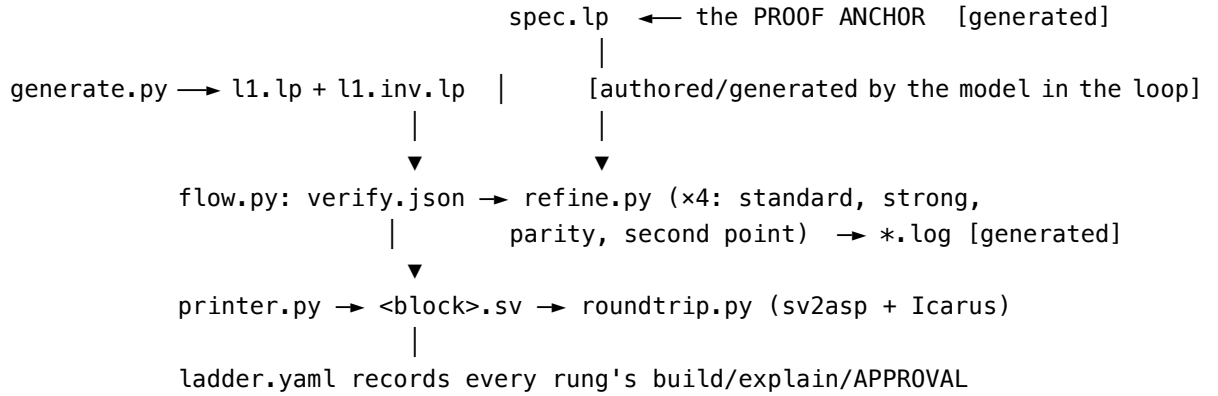
module	its one job
roundtrip.py	print → translate back (the sv2asp translator underneath, modular) → compare authored vs translated traces net-for-net under a scenario, Icarus arbitrating the definite samples
libgen.py	the #script (python) region: every @func the solves need, rendered once, drift-checked against the committed library
export.py, expand.py	the Lean export of data obligations; the expanded (translator-schema) view of an authored design

The language front end (src/sv2asp/aspfirst2/dsl/):

module	its one job
grammar.py	the single source's keeper: slices lib/dsl/grammar.ebnf (surface / core), renders both into the methodology, reports drift
ebnf.py	EBNF → lark, mechanically, at import — the parser cannot drift from the notation people read
expr.py	the core expression parser (lark over the core slice) and the E node tree every consumer walks
parse.py	the structural pass over a core file: declarations, scopes, binders — the shape the checks need
signature.py	the .yaml symbol table's schema: roles, protocols, wire requirements — refusing by name, unknown keys hard errors
cnl.py	the controlled-English desugarer: the sigil check, the pattern matchers <i>compiled from the grammar file's surface section</i> , handlers bound by name, the committed .cnl.core
check.py	the cross-file and semantic checks: names/verbs/directions/fields, scope, lifetime, correspondence, shadowing
emit.py	the compiler's back half : claims → failType monitors at determination instants, behaviours → event + frame monitors, scenarios, the equality theory's concrete half, human-shaped names, the wellformedness guard on its own output
interp.py	the second semantics : the reference interpreter over concrete traces, the random-trace generator, and the differential against the generated contract under clingo

B.3 One entry, end to end — the artifact dataflow





Everything in the right-hand column regenerates under a new tool version; only the authored files must stay valid across upgrades — which is the whole upgrade story.

B.4 How a solve is assembled

Every clingo invocation is: the design's `.lp` (loaded, lane-unrolled), the primitive library (`lib/aspfirst/aspfirst.lp` and companions), the `#script` (python) region from `libgen.py`, the contract, and the leg-specific machinery (the base's reset plan, the step's freed state and hypothesis, a scenario's constrained start). `TOOL.md` walks each phase with what SAT and UNSAT mean there; nothing in that account changed — the language front end sits *before* it and the flow runner *around* it.

B.5 The contracts between the pieces

- **The report is API.** Agents and the flow runner's contains keys parse report lines (INDUCTIVE at K=, NOT EXERCISED, REFUSED, ROUNDTrip: OK). Report strings change additively or loudly (changelog), never quietly.
- **The manifest can select, never weaken.** `verify.json` chooses which checks run and their parameters; what INDUCTIVE means, the dark-read refusal, goals-must-be-reachable are fixed in the tool. Unknown keys are hard errors — a typo silently ignored is a check that silently did not run.
- **Generated is regenerable.** No generated file is ever hand-edited; every one is reproducible from the authored inputs by a command. This is enforced socially by the headers and mechanically wherever a drift gate exists (the core, the grammar's rendered blocks, the committed logs re-derived by the manifest gate on a copy).
- **Refusals are the extension surface.** Anything the subset, the checker, the emitter or the printer cannot handle is a named refusal; extending the tool means turning one refusal into a lowering, with a witness — never widening silently.

Part C — How to: installation, usage, and a first block

C.1 Setting up

You receive **one self-contained folder**. Everything is in it: the tool, the libraries it solves with, this book, the skill a coding agent follows, two worked examples, and a setup script.

Where it goes, and where you work

Put it beside your work, as a **tools** folder:

```
parent/
  tools/          the folder you received -- the tool, the skill, the docs, examples
  myBlock/        YOUR WORKING FOLDER: you run your session here
    SPECIFICATION.md your starting point: the English in force
    myBlock.yaml    then the signature and the controlled English
    myBlock.cnl
    spec.lp  ll.lp  ll.inv.lp  certificate.log  myBlock.sv  ladder.yaml
```

Everything your block produces lives in *your* folder; nothing is ever written into the tools folder. When you start a coding-agent session in myBlock/, point it at ../tools/ — its CLAUDE.md (equivalently AGENTS.md) explains the layout, the rules it must not break, and where the skill and the worked example are.

The short way

```
cd tools
./setup.sh
```

The script assumes **nothing** about your machine — no conda, no named environment, no particular Python on PATH. It finds a Python 3.11 or newer, creates a .venv beside itself (nothing outside the folder is touched), installs the tool and its dependencies into it, checks the two system tools, writes sv2asp.toml pointing at whatever it found, and finishes by running doctor. Two switches: --system-too also installs the missing system tools with your platform's package manager, and --check changes nothing and only reports.

The two system tools, and why one of them is special

clingo is the solver every certificate runs on, and iverilog (Icarus Verilog) arbitrates the round trip. If either is missing, the script prints the exact command for your platform — brew install clingo icarus-verilog on macOS, the distribution's packages on Linux.

The subtlety worth knowing: **clingo must be the executable, and the pip module cannot stand in for it.** Every solve this tool builds carries a #script (python) block — the arithmetic library — which only the command-line clingo executes. doctor probes exactly that, running a small script-carrying program rather than just asking --version, because a clingo without embedded Python passes every naive check and fails every real solve.

If you already have the tools

Then nothing needs installing, and the point is to let the tool *find* them. doctor reports where each one resolved from:

```
$ python -m sv2asp.aspfirst2 doctor
sv2asp2 doctor -- the environment this tool needs
```

```
python      3.11.0 (/opt/homebrew/Caskroom/miniconda/base/envs/logiclab/bin/python)
python: clingo    present
python: pyslang   present
python: lark      present
```

```
python: yaml      present
clingo           clingo version 5.8.0   (/opt/homebrew/bin/clingo)
iverilog         Icarus Verilog version 13.0 (stable) (v13_0)   (/opt/homebrew/bin/iverilog)
clingo: scripts  OK -- the binary runs embedded Python
```

READY -- run a certificate to be sure (see the book's Part C)

If something lives somewhere unusual, put its path in `sv2asp.toml`'s `[tools]` section and re-run `doctor`. Resolution order, first hit wins: an explicit argument → the environment variable (`CLINGO_BIN`, `SV2ASP_PYTHON`, `LEAN_BIN`, `LAKE_BIN`) → that file → `PATH`. The same file is where a design tree declares its own `[funcs]` / `[primitives]` plugins: vendor cells and site-specific arithmetic travel with the *design*, never inside the tool.

Optional extras, needed by no first block: **Verilator** for the few idioms Icarus cannot parse, and **Lean 4 via elan** only when a block exports owed-to-Lean obligations.

Then run something

`setup.sh` builds its `.venv` inside the tools folder, so from your working folder next door the tool is:

```
../tools/.venv/bin/python -m sv2asp.aspfir2 <verb> ...
```

or, once per session, `../tools/.venv/bin/activate` and then simply `sv2asp2 <verb>`. Commands in this book are written as `python -m sv2asp.aspfir2 <verb>`, which is that same command however you have the environment on.

One wrinkle worth knowing in this layout: a `sv2asp.toml` is discovered as `--config PATH` → `$SV2ASP_CONFIG` → `./sv2asp.toml` *in the directory you run from* → `~/.config/sv2asp/config.toml`. `setup.sh` writes one in the tools folder, which your working folder will not see. That matters only if a tool lives somewhere unusual — if `clingo` and `iverilog` are on `PATH`, no config is needed at all. If you do need one, put it in `~/.config/sv2asp/config.toml` (found from anywhere) or export `SV2ASP_CONFIG`.

§C.3 runs a real block end to end.

C.2 Usage — the verbs, in the order a block meets them

verb	what it does
doctor	what the environment has, where each tool resolved from, and the exact command for anything missing — run it first, and again whenever something behaves oddly
ladder init <entry> · ladder status <entry>	open and inspect the seven-rung process; built and explained move a rung, and only a <i>person</i> sets approved (by editing <code>ladder.yaml</code>)
compile <block>.cnl <block>.yaml -o spec.lp	controlled English + signature → the contract, with the mount manifest and every refusal printed
lint <design>.lp	the authoring subset's static checks
refine <spec>.lp <design>.lp --induct K	one certificate run (base + normal-form induction step + scenarios); the linkage <design>.inv.lp is found beside
[--free-reset]	the design

verb	what it does
certificate <entry>	the whole certificate from the entry's verify.json — standard, strong half, parity producers, the second configuration — logs written where the manifest's log keys say
verify <entry>	the same, plus the round trip: the entry's full reproducibility story in one command
print <design>.lp -o <block>.sv	the RTL, under §27.1's conventions
roundtrip <design>.lp <scenario>.lp --icarus	print → translate back → compare value-for-value, Icarus arbitrating

Every verb also takes **--report FILE**, which writes an issue report for the maintainer beside the normal result (§C.4): the tool version, the resolved toolchain, the command, the exit status and the output — and nothing from your design.

Two reading rules that save hours: **a refusal names its construct** — it is the tool being honest about a boundary, not a crash — and **a report's exclusion lines are part of the verdict** (“reset held released”, “NOT EXERCISED”, “bounded-only” bound the claim).

C.3 A block, run for real: the miss queue

The folder carries one complete worked entry at `examples/spec2rtl2/rv_missq/` — an instruction-fetch miss queue with demand and prefetch classes, priority, a fetch limit and a redirect. It is the entry every artifact of the route exists for, built through all seven ladder rungs. Every output below is captured from a real run.

The files, and what each one is (its `README.md` maps them too):

file	what it is
SPECIFICATION.md	the English in force, with a traceability table
rvMissq.yaml	the signature — the compiler's symbol table
rvMissq.cnf	the specification in controlled English — what a person writes
rvMissq.cnf.core	its desugared symbolic core — generated, committed, never hand-edited
spec.lp	the ASP contract, compiled from the two files above
generate.py → l1.lp, l1.inv.lp	the design and its linkage, mounting the contract's windows
verify.json	the flow as data: which checks run, and where their reports go
certificate*.log, CERTIFICATE.md	what was proven, and the raw runner reports
rvMissq.sv	the printed RTL
ladder.yaml	the seven rungs, each built → explained → approved by a person

Run the whole certificate — one command, driven by `verify.json`:

```
$ python -m sv2asp.aspfirst2 certificate examples/spec2rtl2/rv_missq
```

```
flow: examples/spec2rtl2/rv_missq/verify.json
```

```

refine l1.lp: OK
refine l1.lp (induction only): OK
refine l1.lp: OK
second point (depth=2, maxOutstandingFetches=1): OK
second point discriminates: l1.lp rejected, as it must be
FLOW: OK

```

Those six lines are the four runs of §C.2 and their reports land in the entry folder. The first is the standard certificate, the second the strong half with the reset free, the third an independent second contract certifying the same design, and the last pair the off-default configuration — green there, *and* proven to discriminate, because a second point that accepts everything checks nothing.

Read one report. certificate.log says what was actually established:

```

live: OK -- some instant is judged within the base window (the monitors can fire)
base: OK -- from reset, no property can fire on ANY input sequence within 1 live step(s)
induct K=1: state freed at T=0: askPtrHot(0)(1), ... ptrHot(3)(1), ...
induct: no ghost state (no history predicate in the monitors)
induct: reset-exempt monitor(s) NOT EXERCISED in this step (reset held released,
    so they can never fire here): ['emptyUnderReset', 'quietUnderReset'] -- run the
    induction with --free-reset to bind them

```

The **base** says no property can fire on *any* input sequence from reset. The **step** frees every register at $T=0$ — the induction is over all reachable states, not a simulation — and “no ghost state” says the contract watches the design through windows rather than a mirror of it. The **NOT EXERCISED** line is the runner refusing a vacuous claim: those two monitors are judged only where reset is asserted, this step pins reset released, so they are excluded here and bound by the strong half instead. Together with the base, the rest hold for all time.

Run the round trip (the certificate’s sibling: does the printed RTL agree?):

```

$ python -m sv2asp.aspfirst2 roundtrip \
    examples/spec2rtl2/rv_missq/l1.lp \
    examples/spec2rtl2/rv_missq/roundtrip_scenario.lp --icarus

```

```

icarus agrees on all 2679 definite samples
ROUNDTrip: OK

```

Three independent readings of one story — the authored logic, the printed SystemVerilog translated back by a separate translator, and Icarus — agreeing on every signal at every cycle. `verify` runs this and the certificate together.

The other example, `examples/rtl2asp/fsm_demo/`, is the translator underneath: a small SystemVerilog FSM and the ASP it becomes. You never invoke that translator directly — the round trip does — but reading `fsm.sv` beside `fsm.lp` is the quickest way to see what the model the proofs run on actually looks like.

Where to go from here: the skill (`.claude/skills/spec2rtl-dsl/SKILL.md`) is the operating procedure for building an entry of your own, and the methodology (`ROUTE_METHODOLOGY.md`) is the book behind the route — Part II is the language reference you write a `.cnl` against.

C.4 Upgrades, and reporting a gap

The tool is maintained centrally. You install it; you never patch it. That is not bureaucracy: a certificate produced by a modified tool is a certificate about nothing, and the trust chain is the product.

When a new version arrives. Your *authored* files — the signature, the controlled English, the design — are what carry across; everything generated (the core, the contract, the printed RTL, the logs) is regenerated by the tool. So an upgrade is:

```
pip install --upgrade <the new engine>          # and unzip the matching workspace bundle
python -m sv2asp.aspfir2 verify blocks/<yourBlock>
```

verify replays the whole flow and re-proves the block under the new version. Two things follow, both intended: regenerated artifacts change their digests, so the ladder marks those rungs stale and a person re-approves what the new tool produced; and a regression shows up as a named failure in your own folder rather than as silent drift. The engine and the workspace bundle share a version — the skill and this book *describe* the tool's behaviour, so a mismatched pair misleads you (and misleads an agent worse).

When the tool refuses something it should accept. That is the normal way a gap surfaces — every boundary is a named refusal rather than a silent wrong. Re-run the command with `--report`, which every verb accepts:

```
python -m sv2asp.aspfir2 compile myBlock.cnl myBlock.yaml -o spec.lp --report issue.txt
```

The verdict is unchanged — a refusal still refuses, with the same exit status — and `issue.txt` is written beside it carrying exactly what a diagnosis needs: the tool version, the **resolved** toolchain (which clingo, from where, at what version — the question nobody answers reliably from memory), the command, the exit status, and the tool's own output. It carries nothing from your design; attach a *minimised probe* if you can make one, but never your block. Your design is yours, exactly as the tool's internals are the maintainer's. A fix comes back as a new version, with the case added to the gates in Part A.